Decisiones del administrador que afectan a la base de datos

Sistema Zaca · 25 de octubre de 2026 · Para el compañero de base de datos · Basado en los scripts V6 y V7

La aplicación (JavaFX) ya tiene funcionales las cinco secciones del administrador: Rutas, Unidades, Taquillas, Personal y Cuentas. En todas se siguió el script tal como está. Este documento lista lo que la aplicación **asume o decide por su cuenta** y lo que se necesita saber o cambiar en la base. Las filas con **Riesgo** son las que conviene revisar primero.

1. Qué tablas usa cada sección

Sección	Tablas	Operaciones
Rutas	viajes.ruta, viajes.ruta_paradas, viajes.precio_tramo, viajes.tipo_servicio, administracion.terminal (solo lee)	INSERT y UPDATE de ruta; DELETE e INSERT de ruta_paradas; INSERT de precio_tramo, todo en una transacción
Unidades	viajes.unidad, viajes.tipo_bus, viajes.nombre_tipo_bus, viajes.layout	INSERT y UPDATE de unidad; busca o crea el tipo_bus
Taquillas	administracion.taquilla, administracion.terminal, administracion.estado_conexion	INSERT y UPDATE de taquilla; INSERT de terminal (nueva ciudad)
Personal	administracion.personal, administracion.puesto, administracion.personal_app (solo EXISTS)	INSERT, UPDATE y DELETE de personal
Cuentas	administracion.personal_app, administracion.rol, personal, terminal, taquilla	INSERT y UPDATE de personal_app (incluye password_hash)

2. Decisiones por sección

Rutas

#	Qué hace la aplicación	Qué dice el script / riesgo	Qué se pide a la base
R1	El *orden* de las paradas empieza en 1 y es contiguo (1, 2, 3…). *precio_tramo* usa esas posiciones.	El script no dice si empieza en 0 o en 1.	Confirmar la convención. Si prefieren otra, se cambia en un solo lugar.
R2	Exige mínimo 2 paradas.	No está en el script (una ruta de 1 parada no tiene tramos que cobrar).	Opcional: validarlo en la base si lo consideran regla de negocio.
R3	Al editar una ruta borra todas sus paradas y las vuelve a insertar; los precios se borran en cascada.	**Riesgo:** si la ruta ya tiene viajes y boletos, estos guardan órdenes de parada y podrían descuadrarse. El script no lo impide.	Decidir la regla (el contrato proponía RUTA_EN_USO, Z0008): impedir cambiar paradas con viajes, o permitirlo.
R4	No exige que km y minutos vayan en aumento ni que la primera parada sea 0.	El script solo pide >= 0 y orden único.	Si lo quieren, agregarlo como CHECK o trigger.
R5	Acepta precios de 0 o más.	Igual que el script (precio >= 0).	Ninguna.

Unidades

#	Qué hace la aplicación	Qué dice el script / riesgo	Qué se pide a la base
U1	Al guardar una unidad, busca el *tipo_bus* con ese tipo, acomodo y capacidad; si no existe, lo crea.	*tipo_bus* no tiene datos de inicio. No hay UNIQUE sobre (tipo, acomodo, capacidad), así que podrían duplicarse combinaciones.	Elegir: insertar tipos de autobús fijos en el script, o dejar la creación dinámica y agregar ese UNIQUE.

#	Qué hace la aplicación	Qué dice el script / riesgo	Qué se pide a la base
U2	La pantalla **no muestra** el estado "En taller".	Viene de *taller_db* por postgres_fdw, que no está en V6 ni V7.	Una vista que una unidad con su estado de mantenimiento. Cuando exista, se agrega la columna.

Taquillas y terminales

#	Qué hace la aplicación	Qué dice el script / riesgo	Qué se pide a la base
T1	Una terminal nueva ("Nueva ciudad") se guarda con estado **DESCONECTADO**, buscado por nombre.	*terminal.id_conexion* es NOT NULL y no tiene valor por defecto.	Poner DEFAULT en id_conexion, o confirmar que DESCONECTADO es el estado inicial correcto.
T2	Permite dos terminales con el mismo nombre.	No hay UNIQUE sobre el nombre.	Opcional: UNIQUE sobre lower(nombre_terminal) (lo pedía el contrato).
T3	Pide folios enteros mayores que 0.	El script solo exige rango_inicial <= rango_final.	Ninguna, salvo que quieran el CHECK > 0.
T4	Permite cambiar la serie o el folio inicial aunque ya haya folios usados, y un folio final menor que los usados.	**Riesgo:** repetir números de ticket. El script no lo impide.	Trigger que lo bloquee (el contrato proponía FOLIOS_EMITIDOS, Z0009).
T5	*folios_usados* solo se muestra; la aplicación no lo modifica.	Lo debe actualizar la venta.	Confirmar quién lo incrementa al vender.
T6	Para distinguir "serie repetida" de "identificador repetido" lee el nombre de la restricción en el mensaje (*serie_folio*).	Depende del nombre generado por PostgreSQL (taquilla_serie_folio_key).	No renombrar esa restricción, o avisar si cambia.

Personal

#	Qué hace la aplicación	Qué dice el script / riesgo	Qué se pide a la base
P1	Pasa la CURP a mayúsculas y valida con el mismo patrón del script (sexo solo H o M).	La CURP de *ventas.pasajero* sí acepta X; la de *personal* no.	Unificar los dos patrones.
P2	Licencia y vigencia son opcionales y solo se piden a los choferes (campo habilitado según el puesto). Muestra un aviso si vence en 30 días o ya venció.	El script las deja opcionales aun para CONDUCTOR.	Opcional: CHECK que las exija si el puesto es CONDUCTOR.
P3	Eliminar es un **DELETE real**; borra también la cuenta (cascada). Si es chofer con viajes, la base lo rechaza (23503) y se avisa.	No existe columna "activo" en *personal*, así que no hay baja lógica. Borrar a alguien puede afectar el historial.	**Decidir:** agregar *activo* (baja lógica) o mantener el borrado real. Pendiente desde el contrato.
P4	Teléfono opcional (se guarda NULL si está vacío); si se escribe, 10 dígitos.	Igual que check_telefono.	Ninguna.

Cuentas

#	Qué hace la aplicación	Qué dice el script / riesgo	Qué se pide a la base
C1	Guarda la contraseña con BCrypt (jbcrypt, formato $2a$), igual que la cuenta existente. **Sin largo mínimo.**	El script solo exige que password_hash no sea nulo.	Ninguna. Quien valide contraseñas debe usar BCrypt.

#	Qué hace la aplicación	Qué dice el script / riesgo	Qué se pide a la base
C2	El usuario se genera solo: inicial del nombre + apellido paterno sin acentos, con número si se repite.	El script solo exige que sea único.	Ninguna.
C3	Exige una sede (id_terminal) al crear la cuenta.	En V7 *id_terminal* es opcional.	Confirmar que debe ser obligatoria; si sí, NOT NULL.
C4	Un taquillero puede tener una **taquilla asignada** (id_taquilla), de la sede elegida. Es opcional.	El diseño usaba "turnos"; el script no tiene tabla de turnos, solo id_taquilla.	Confirmar que la taquilla fija por cuenta reemplaza al turno.
C5	No valida el rol contra el puesto: un chofer puede quedar con rol de administrador si se elige a mano.	No hay relación entre puesto y rol en el script.	Opcional: regla en la base.
C6	Desactivar solo cambia *activo*; la cuenta no se borra. La regla "no desactivarse a sí mismo" queda pendiente.	Necesita el login real (aún no existe).	Ninguna por ahora.

3. Conexión y funcionamiento general

#	Qué hace la aplicación	Qué dice el script / riesgo	Qué se pide a la base
I1	Una sola conexión para toda la aplicación (Singleton), hacia **Pgpool-II**. Se configura en un archivo .env (URL, usuario y contraseña); el ejemplo usa el puerto 9999.	Si la conexión se cae o Pgpool reinicia, abre una nueva sola.	Confirmar el puerto de Pgpool y el usuario que debe usar la aplicación.
I2	Los guardados de varias sentencias van en una transacción (autocommit apagado). Después de guardar, la pantalla vuelve a leer la lista de inmediato.	Con replicación streaming, esa lectura podría caer en la réplica y verse atrasada.	Revisar en Pgpool que las lecturas justo después de una escritura vayan al primario (parámetro disable_load_balance_on_write) o el retraso de la réplica.
I3	Busca algunos catálogos por **nombre**, no por id: DESCONECTADO, CONDUCTOR, TAQUILLERO, DESPACHADOR, ADMINISTRADOR, MECANICO.	Si cambian esos nombres, la aplicación deja de funcionar bien.	No renombrar esos valores sin avisar.
I4	Traduce errores por código SQLSTATE: 23505 (repetido), 23514 (CHECK) y 23503 (llave foránea), con mensajes propios.	Los demás errores los muestra con el texto de la base.	Mantener esos códigos al agregar reglas nuevas.
I5	Hoy todo es SQL directo en los DAO. Es temporal, igual que en la biblioteca.	Candidatos a vista o procedimiento: lista de rutas con sus paradas y precios, guardado de ruta, y lista de personal.	Cuando quieran, se escriben y los DAO solo los llaman.
I6	Las pruebas crearon y borraron filas de prueba en la base local (nombres que empiezan con ZZ).	Las secuencias de identidad avanzaron; no quedó ningún dato de prueba.	Ninguna.

4. Lo que se necesita decidir primero

Riesgo para datos: R3 (editar paradas de una ruta con viajes), T4 (cambiar serie o folios con folios usados) y P3 (borrado real de personal).
Para poder avanzar: U1 (tipos de autobús), T1 (estado de las terminales nuevas), U2 (vista con taller_db) e I2 (lecturas después de escribir con Pgpool).
Opcionales: R2, R4, T2, T3, P1, P2, C3, C4 y C5 son reglas de negocio que la base puede reforzar si lo consideran necesario.